Practice 1.1

Aim: Write a C program to find total, average of n students using structures

Description:

In C, structures are a way to group different types of variables under a single name. A structure can hold variables of different data types, including arrays, pointers, and other structures. This is particularly useful when you need to represent a record, such as a student or an employee, where each record may contain different types of information (name, age, salary, etc.).

Defining a Structure

You can define a structure using the struct keyword.

```
struct Person {
    char
    name[50];
    int age;
    float salary;
};
```

Array of structures:

An array of structures in programming is a collection where each element of the array is a structure. A structure (often referred to as a struct) is a user-defined data type that groups different data types together. This is useful when you want to represent a record that holds various types of data under one name.

ADITYA UNIVERSITY

Program:

```
#include
<stdio.h> struct
Student
 {

    char name[50];
    int m1,m2,m3;  // Assuming each student has 3 subjects float
    total,avg;
};

int main()
 {

    int n, i;
    printf("Enter the number of students: ");
    scanf("%d", &n);
    struct Student s[n];
    // Input data for each student
    for(i = 0; i < n; i++)
        {

            printf("enter name,3 subject marks of student %d\n",i+1);
            scanf("%s%d%d%d",s[i].name,&s[i].m1,&s[i].m2,&s[i].m3);
        }
```

```
    for(i=0;i<n;i++)
    {
        s[i].total=s[i].m1+s[i].m2+s[i].m3; s[i].avg=s[i].total/3.0;
        printf("Total and Average marks of Student %d=%.2f,%.2f\n",i+1,s[i].total,s[i].avg);
    }

    return 0;
}
```

OUTPUT:

Enter the number of students: 2
enter name,3 subject marks of student 1 Aditya
56 78 57
enter name,3 subject marks of student 2 Vamc
99 98 67
Total and Average marks of Student 1=191.00,63.67 Total
and Average marks of Student 2=264.00,88.00

ADITYA UNIVERSITY

Practice 1.2

AIM: Write a C Program copy one structure variable to another structure of the same type

Description:

A **structure assignment** typically refers to the operation of assigning values between structures or copying the contents of one structure to another. This can be done directly, using the assignment operator (=), if the structures are of the same type. In a structure assignment, you transfer the values of all fields from one structure to another.

Program:

```
#include<stdio.h
> struct student
{
        char
        name[15]; int
        rno;
        char gender;
};

int main()
{
        struct student s1={"Srinu",143,'m'},s2; s2=s1;
        printf("The Student Details are:\n");
        printf("Name:%s\nRno=%d\nGender=%c",s2.name,s2.rno,s2.gender); return 0;
}
```

OUTPUT:

The Student Details are:
Name: Aditya
Rno=143
Gender=m

Practice 1.3

AIM: Write a C Program read student name marks from the command line and display the student details along with total

DESCRIPTION:

Command Line Arguments in C allow users to pass information to a program when executing it from the command line or terminal. These arguments can be accessed inside the main() function via the parameters argc (argument count) and argv (argument vector).

PROGRAM:

```
#include<stdio.h>
#include<string.h>
struct student
{
        char name[10];
        int
        s1,s2,s3,s4,s5;
};

int main(int argc,char *argv[])
{
        struct student
        s; int total;
        strcpy(s.name,argv[1]);
        s.s1=atoi(argv[2]);
        s.s2=atoi(argv[3]);
        s.s3=atoi(argv[4]);
        s.s4=atoi(argv[5]);
        s.s5=atoi(argv[6]); total=s.s1+s.s2+s.s3+s.s4+s.s5;
        printf("name\ts1\ts2\ts3\ts4\ts5\total\n");
        printf("%s\t%d\t%d\t%d\t%d\t%d\t%d",s.name,s.s1,s.s2,s.s3,s.s4,s.s5,total); return 0;
}
```

OUTPUT:

```
C:\Users\DS Programs>arg.exe Aditya 56 67 89 90 87
name      s1      s2      s3      s4      s5      total
Aditya    56      67      89      90      87      389
```

Practice 2.1

AIM: Write a C Program Implement Merge sort using arrays

DESCRIPTION:

Algorithm Overview:

Merge Sort is a sorting technique that follows the *divide and conquer* approach. The main idea is to divide the given unsorted array into smaller sub-arrays until each sub-array has only one element (which is inherently sorted), and then merge these sub-arrays in a sorted manner to produce the final sorted array.

Steps of the Algorithm:

1. **Divide** the array into two halves.
2. **Recursively** sort both halves.
3. **Merge** the two sorted halves into one sorted array

```
MergeSort(arr, left, right):
  if left < right:
    mid = (left + right) / 2
    MergeSort(arr, left, mid)
    MergeSort(arr, mid+1, right)
    Merge(arr, left, mid, right)
```

ADITYA UNIVERSITY

Merge Function:

- Takes two sorted sub-arrays and combines them into a single sorted array.
- Compares elements from both sub-arrays and arranges them in order.

Time Complexity:

- Best Case: O(n log n)
- Average Case: O(n log n)
- Worst Case: O(n log n)

Space Complexity:

- O(n) – due to the temporary arrays used during the merging process.

Advantages:

- Stable sort (maintains the relative order of equal elements).
- Consistent performance even in the worst case.

Disadvantages:

- Requires additional space.
- Recursive, which may lead to stack overflow for large inputs

PROGRAM:

```
#include <stdio.h>

// Function declarations
void divide(int arr[], int l, int r);
void merge(int arr[], int l, int m, int r);

int n;

int main()
 {
   int arr[10], i;

   // Input number of elements
   printf("Enter the number of elements to be sorted: ");
   scanf("%d", &n);

   // Input array elements
   printf("Enter %d elements to be sorted:\n", n);
   for (i = 0; i < n; i++) {
      printf("Arr[%d] = ", i);
      scanf("%d", &arr[i]);
   }

   // Call merge sort
   divide(arr, 0, n - 1);

   // Print sorted array
   printf("\nSorted array is:\t");
   for (i = 0; i < n; i++) {
      printf("%4d", arr[i]);
   }
   printf("\n");

   return 0;
}

// Recursive function to divide the array
```

ADITYA UNIVERSITY

```
void divide(int arr[], int l, int r) {
    if (l < r) {
        int m = (l + r) / 2;

        // Divide left half
        divide(arr, l, m);

        // Divide right half
        divide(arr, m + 1, r);

        // Merge the sorted halves
        merge(arr, l, m, r);
    }
}

// Function to merge two sorted sub-arrays
void merge(int arr[], int l, int m, int r) {
    int i, j, k;
    int n1 = m - l + 1; // Size of left sub-array
    int n2 = r - m;     // Size of right sub-array

    // Temporary arrays
    int L[n1], R[n2];

    // Copy data to temporary arrays
    for (i = 0; i < n1; i++)
        L[i] = arr[l + i];
    for (j = 0; j < n2; j++)
        R[j] = arr[m + 1 + j];

    i = 0; j = 0; k = l;

    // Merge the temp arrays back into arr[l..r]
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

    // Copy remaining elements of L[], if any
```

ADITYA UNIVERSITY

```
    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }

    // Copy remaining elements of R[], if any
    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }
}
```

OUTPUT:

Enter the number of elements to be sorted: 5
Enter 5 elements to be sorted:
Arr[0] = 20
Arr[1] = 10
Arr[2] = 5
Arr[3] = 7
Arr[4] = 1

Sorted array is: 1 5 7 10 20

ADITYA UNIVERSITY

Practice 2.2

AIM: Write a C Program to Implement Quick sort using arrays

DESCRIPTION:

Quick Sort is an efficient, in-place, divide-and-conquer sorting algorithm. It works by selecting a *pivot* element from the array and partitioning the other elements into two sub-arrays—according to whether they are less than or greater than the pivot. The sub-arrays are then sorted recursively.

Steps of the Algorithm:

1. **Choose a pivot** (commonly the first, last, or middle element).
2. **Partition** the array:
   - Place elements smaller than the pivot to the left.
   - Place elements greater than the pivot to the right.
3. **Recursively apply** the same logic to the left and right sub-arrays.

Time Complexity:

- **Best Case**: O(n log n)
- **Average Case**: O(n log n)
- **Worst Case**: $O(n^2)$ (occurs when the smallest or largest element is always chosen as the pivot)

Space Complexity:

ADITYA UNIVERSITY

- O(log n) (due to recursive calls; in-place sorting)

Advantages:

- Faster in practice than many other sorting algorithms (e.g., merge sort) for small to medium datasets.
- In-place sorting (does not require additional memory).

Disadvantages:

- Not stable (may change the relative order of equal elements).
- Performance degrades to $O(n^2)$ in the worst case (e.g., already sorted data with poor pivot choice).

PROGRAM:

```
#include <stdio.h>

// Function declarations
void quicksort(int [], int, int);
int partition(int [], int, int);

int main() {
    int a[50], n, i;

    // Input number of elements
    printf("Enter the number of elements: ");
    scanf("%d", &n);

    // Input array elements
    printf("\nEnter the %d elements to be sorted:\n", n);
    for (i = 0; i < n; i++) {
        printf("a[%d] = ", i);
        scanf("%d", &a[i]);
    }

    // Call quicksort
    quicksort(a, 0, n - 1);

    // Output sorted array
    printf("\nArray after applying Quick Sort:\n");
    for (i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");

    return 0;
}

// Recursive Quick Sort function
void quicksort(int a[], int low, int high) {
    if (low < high) {
        int j = partition(a, low, high);
        quicksort(a, low, j - 1);
        quicksort(a, j + 1, high);
    }
}
```

ADITYA UNIVERSITY

```
// Partition function
int partition(int a[], int low, int high) {
    int pivot = a[low]; // First element as pivot
    int i = low + 1;
    int j = high;
    int temp;

    while (i <= j) {
        while (i <= high && a[i] <= pivot) {
            i++;
        }
        while (a[j] > pivot) {
            j--;
        }
        if (i < j) {
            // Swap a[i] and a[j]
            temp = a[i];
            a[i] = a[j];
            a[j] = temp;
        }
    }

    // Swap pivot with a[j]
    temp = a[low];
    a[low] = a[j];
    a[j] = temp;

    return j; // Return partition index
}
```

ADITYA UNIVERSITY

OUTPUT:

Enter the number of elements: 4

Enter the 4 elements to be sorted:
a[0] = 5
a[1] = 1
a[2] = 8
a[3] = 2

Array after applying Quick Sort:
1 2 5 8

Practice 2.3

AIM: Write a C Program to Implement Radix Sort using arrays

DESCRIPTION:

Radix Sort is a **non-comparative** sorting algorithm that sorts numbers by processing **individual digits**. It works by sorting numbers digit by digit, starting from the **least significant digit (LSD)** to the **most significant digit (MSD)** using a **stable sorting technique** like counting sort as a subroutine.

Steps of the Algorithm:

1. Find the **maximum number** to determine the number of digits.
2. Starting from the **least significant digit**, sort the array using a **stable sort** (e.g., counting sort).
3. Repeat the process for each digit position (units, tens, hundreds, etc.).
4. After the final pass, the array is sorted.

Pseudocode:

```
RADIXSORT(arr, n):
max = find maximum number in arr
exp = 1
while max / exp > 0:
 COUNTINGSORT(arr, n, exp)
 exp = exp * 10
```

ADITYA UNIVERSITY

Advantages:

- Efficient for sorting large numbers with small digit lengths.
- Performs better than comparison-based sorts (like quick sort) for specific cases.

Disadvantages:

- Works only for integers or fixed-length strings.
- Requires extra space for counting arrays or buckets.
- Not suitable for floating-point numbers or large-digit values.

Time Complexity:

- **Best Case**: O(nk)
- **Average Case**: O(nk)
- **Worst Case**: O(nk)

PROGRAM:

```
#include <stdio.h>

// Function to get the maximum value in the array
int getMax(int arr[], int n) {
    int max = arr[0];
    for (int i = 1; i < n; i++)
        if (arr[i] > max)
            max = arr[i];
    return max;
}

// Function to perform Counting Sort based on a specific digit (exp)
void countingSort(int arr[], int n, int exp) {
    int output[n];  // Output array
    int count[10] = {0};

    // Count occurrences of each digit
    for (int i = 0; i < n; i++)
        count[(arr[i] / exp) % 10]++;

    // Update count[i] so that it contains actual position of the digit in output[]
    for (int i = 1; i < 10; i++)
        count[i] += count[i - 1];

    // Build the output array (stable sort)
    for (int i = n - 1; i >= 0; i--) {
        output[count[(arr[i] / exp) % 10] - 1] = arr[i];
        count[(arr[i] / exp) % 10]--;
    }

    // Copy the sorted output array to arr[]
    for (int i = 0; i < n; i++)
        arr[i] = output[i];
}

// Main Radix Sort function
void radixSort(int arr[], int n) {
    int max = getMax(arr, n);

    // Apply counting sort to sort elements based on each digit
    for (int exp = 1; max / exp > 0; exp *= 10)
        countingSort(arr, n, exp);
}
```

ADITYA UNIVERSITY

```
int main() {
    int arr[100], n;

    // Input array size and elements
    printf("Enter the number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements:\n", n);
    for (int i = 0; i < n; i++) {
        printf("arr[%d] = ", i);
        scanf("%d", &arr[i]);
    }

    // Call radix sort
    radixSort(arr, n);

    // Display sorted array
    printf("\nSorted array:\n");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");

    return 0;
}
```

ADITYA UNIVERSITY

OUTPUT:

```
Enter the number of elements: 5
Enter 5 elements:
arr[0] = 4
arr[1] = 1
arr[2] = 9
arr[3] = 5
arr[4] = 2

Sorted array:
1 2 4 5 9
```

Practice 3.1

AIM: Write a C Program to implement Single Linked List and it's operations.

DESCRIPTION:

A **Singly Linked List** is a dynamic data structure consisting of a sequence of elements called **nodes**. Each node stores:

- **Data**: The actual value.
- **Link** (or pointer): The address of the next node in the list.

Unlike arrays, linked lists do not require contiguous memory locations and can grow or shrink during execution.

Components of a Node:

```
struct Node {
   int data;
   struct Node* next;
};
```

- data holds the value.
- next is a pointer to the next node.

Operations on Singly Linked List:

1. Create List

- Allocate memory for new nodes using malloc or new.
- Link the nodes using pointers.
- The list starts from the **head** pointer.

2. Insert Node

- **At Beginning**:
  Create a new node, point its next to the current head, and update the head.
- **At End**:
  Traverse to the last node and link its next to the new node.
- **At Specific Position**:
  Traverse to the node before the desired position, and adjust the pointers accordingly.

3. Delete Node

- **From Beginning**:
  Point the head to the second node and free the first.
- **From End**:
  Traverse to the second last node, set its next to NULL, and free the last node.
- **From Specific Position**:
  Adjust the links to bypass the node to be deleted and free its memory.

4. Display/Traverse List

- Start from the head and move through each node using the next pointer until NULL is reached, printing each node's data.

5. Search Element

- Traverse the list and compare each node's data with the target value.

6. Count Nodes

- Traverse the list and increment a counter until the end is reached.

Advantages:

- Dynamic size (no need to define size beforehand).
- Easy insertion and deletion of nodes.

Disadvantages:

- No random access to elements.

ADITYA UNIVERSITY

PROGRAM:

```
#include<stdio.h>
#include<stdlib.h>
void create_list();
void display();
void insert_at_beginning();
void insert_at_ending();
void insert_at_position();
void deletion_at_begining();
void deletion_at_ending();
void deletion_at_position();
struct node
{
	int data;
	struct node *next;
};
struct node *head;
int main()
{
	display();
	int choice;
	while(1)
	{
	printf("\n1.create_list\n2.insert_at_beginning\n3.insert_at_ending\n4.insert_at_position\n5.di
aplay\n6.deletion_at_begining\n7.deletion_at_ending\n8.deletion_at_position\n9.exit\n");
	printf("enter choice:\n");
	scanf("%d",&choice);
	switch(choice)
	{
		case 1:create_list();
		break;
		case 2:insert_at_beginning();
		break;
		case 3:insert_at_ending();
		break;
		case 4:insert_at_position();
		break;
		case 5:display();
		break;
		case 6:deletion_at_begining();
		break;
		case 7:deletion_at_ending();
		break;
		case 8:deletion_at_position();
```

ADITYA UNIVERSITY

```
                break;
                case 9:exit(0);
                default:
                        printf("invalid choice!");
        }
    }
        return 0;
}
void create_list()
{
    struct node *first,*second,*n1,*n2;
        first=(struct node*)malloc(sizeof(struct node));
        second=(struct node*)malloc(sizeof(struct node));
        n1=(struct node*)malloc(sizeof(struct node));
        n2=(struct node*)malloc(sizeof(struct node));
        printf("enter 4 values:\n");
        scanf("%d",&first->data);
        first->next=second;
        scanf("%d",&second->data);
        scanf("%d%d",&n1->data,&n2->data);
        second->next=n1;
        n1->next=n2;
        n2->next=NULL;
        head=first;
        display();
}
void display()
{
        struct node *temp=head;
        while(temp!=NULL)
        {
                printf("%d-->%d\n",temp->data,temp->next);
                temp=temp->next;
        }
}
void insert_at_beginning()
{
        struct node *new;
        new=(struct node*)malloc(sizeof(struct node*));
        printf("enter new node value:\n");
        scanf("%d",&new->data);
        new->next=head;
        head=new;
        display();
}
```

ADITYA UNIVERSITY

```
void insert_at_ending()
{
        struct node *new1;
        new1=(struct node*)malloc(sizeof(struct node*));
        printf("enter new1 node value:\n");
        scanf("%d",&new1->data);
        new1->next=NULL;
        struct node *temp=head;
        while(temp->next!=NULL)
        {
                temp=temp->next;
        }
        temp->next=new1;
        display();
}
void insert_at_position()
{
        int pos,i;
        printf("enter position:\n");
        scanf("%d",&pos);
        struct node *new2;
        new2=(struct node*)malloc(sizeof(struct node*));
        printf("enter new2 node value:\n");
        scanf("%d",&new2->data);
        struct node *temp=head;
        for(i=1;i<pos;i++)
        {
                if(temp->next!=NULL)
                {
                        temp=temp->next;
                }
        }
        new2->next=temp->next;
        temp->next=new2;
        display();
}
void deletion_at_begining()
{
        if(head==NULL)
        return;
        struct node *temp=head;
        head=head->next;
        free(temp);
        display();
}
```

ADITYA UNIVERSITY

```
void deletion_at_ending()
{
        if (head == NULL)
   return;
   if (head->next == NULL)
        {
      free(head);
      head = NULL;
      return;
   }
   struct node* temp = head;
   while (temp->next != NULL && temp->next->next != NULL)
        {
      temp = temp->next;
   }
   struct node *lastNode = temp->next;
   temp->next = NULL;
   free(lastNode);
   display();
}
void deletion_at_position()
{
        struct node *temp,*nextnode;
        int loc,i=1;
        printf("enter loc:\n");
        scanf("%d",&loc);
        temp=head;
        while(i<loc-1)
        {
                temp=temp->next;
                i=i+1;
        }
        nextnode=temp->next;
        temp->next=nextnode->next;
        free(nextnode);
        display();
}
```

OUTPUT:

1.create_list
2.insert_at_beginning
3.insert_at_ending
4.insert_at_position

```
5.diaplay
6.deletion_at_begining
7.deletion_at_ending
8.deletion_at_position
9.exit
enter choice:
1
enter 4 values:
7 8 9 4
7-->7498720
8-->7498752
9-->7498784
4-->0
1.create_list
2.insert_at_beginning
3.insert_at_ending
4.insert_at_position
5.diaplay
6.deletion_at_begining
7.deletion_at_ending
8.deletion_at_position
9.exit
enter choice:
2
enter new node value:
6
6-->7498688
7-->7498720
8-->7498752
9-->7498784
4-->0
1.create_list
2.insert_at_beginning
3.insert_at_ending
4.insert_at_position
5.diaplay
6.deletion_at_begining
7.deletion_at_ending
8.deletion_at_position
9.exit
enter choice:
3
enter new1 node value:
1
6-->7498688
```

ADITYA UNIVERSITY

```
7-->7498720
8-->7498752
9-->7498784
4-->7498848
1-->0
1.create_list
2.insert_at_beginning
3.insert_at_ending
4.insert_at_position
5.diaplay
6.deletion_at_begining
7.deletion_at_ending
8.deletion_at_position
9.exit
enter choice:
4
enter position:
2
enter new2 node value:
3
6-->7498688
7-->7498880
3-->7498720
8-->7498752
9-->7498784
4-->7498848
1-->0
1.create_list
2.insert_at_beginning
3.insert_at_ending
4.insert_at_position
5.diaplay
6.deletion_at_begining
7.deletion_at_ending
8.deletion_at_position
9.exit
enter choice:
6
7-->7498880
3-->7498720
8-->7498752
9-->7498784
4-->7498848
1-->0
1.create_list
```

ADITYA UNIVERSITY

```
2.insert_at_beginning
3.insert_at_ending
4.insert_at_position
5.diaplay
6.deletion_at_begining
7.deletion_at_ending
8.deletion_at_position
9.exit
enter choice:
7
7-->7498880
3-->7498720
8-->7498752
9-->7498784
4-->0
1.create_list
2.insert_at_beginning
3.insert_at_ending
4.insert_at_position
5.diaplay
6.deletion_at_begining
7.deletion_at_ending
8.deletion_at_position
9.exit
enter choice:
8
enter loc:
3
7-->7498880
3-->7498752
9-->7498784
4-->0
1.create_list
2.insert_at_beginning
3.insert_at_ending
4.insert_at_position
5.diaplay
6.deletion_at_begining
7.deletion_at_ending
8.deletion_at_position
9.exit
enter choice:
5
7-->7498880
3-->7498752
```

ADITYA UNIVERSITY

9-->7498784
4-->0
1.create_list
2.insert_at_beginning
3.insert_at_ending
4.insert_at_position
5.diaplay
6.deletion_at_begining
7.deletion_at_ending
8.deletion_at_position
9.exit
enter choice:
9

ADITYA UNIVERSITY

Practice 3.2

AIM: Write a C Program to perform reversing operation on single linked list

DESCRIPTION:

Reversing a singly linked list involves changing the direction of the next pointers of all nodes. After reversal:

- The head pointer should point to the original last node.
- Each node's next should point to the previous node instead of the next one.

Since a singly linked list only has one pointer (next), we need to **manipulate the links carefully** without losing access to the rest of the list

Approach:

We use three pointers:

- prev – Initially set to NULL
- current – Points to the current node being processed
- next – Temporarily stores the next node before changing the link

Steps:

1. Initialize

ADITYA UNIVERSITY

```
struct Node* prev = NULL;
struct Node* current = head;
struct Node* next = NULL;
```

2.Traverse the list:

- Store the next node: next = current->next
- Reverse the link: current->next = prev
- Move prev and current one step forward:

  prev = current;

  current = next;

3.After the loop, set head = prev

PROGRAM:

```
#include<stdio.h>
#include<stdlib.h>
struct node
{
int data;
struct node *next;
};
typedef struct node NODE; NODE *head=NULL;
 NODE *create();
void traverse(); void insert();
void reverse(NODE*);
NODE *create()
{
NODE *newnode=(NODE*)malloc(sizeof(NODE));
 printf("Enter Data:");
scanf("%d",&newnode->data);
newnode->next=NULL;
return newnode;
}
void traverse()
{
NODE *t=head;
 while(t!=NULL)
{
printf("-->%d",t->data);
 t=t->next;
}
printf("\n");
}
void insert()
{
NODE *newnode=create(),*t;
if(head==NULL)
 head=newnode;
else
{
t=head;
while(t->next!=NULL)
 t=t->next;
t->next=newnode;
}

}
```

ADITYA UNIVERSITY

```
void reverse(NODE *t)
{
if(t==NULL)
return; else
{
reverse(t->next);
printf("-->%d",t->data);
}
}

int main()
{
int n,i;
printf("Enter Number of nodes to insert:");
 scanf("%d",&n);
for(i=0;i<n;i++) insert();
printf("Before reverse\n");
 traverse();
printf("After reverse\n");
reverse(head);
return 0;
}
```

ADITYA UNIVERSITY

OUTPUT:

```
Enter Number of nodes to insert:10
Enter Data:10
Enter Data:20
Enter Data:30
Enter Data:40
Enter Data:50
Enter Data:60
Enter Data:70
Enter Data:80
Enter Data:90
Enter Data:100
Before reverse
-->10-->20-->30-->40-->50-->60-->70-->80-->90-->100
After reverse
-->100-->90-->80-->70-->60-->50-->40-->30-->20-->10
```

Practice 3.3

AIM: Write a C Program to implement Double Linked List and it's operations

DESCRIPTION:

Definition:

A **Doubly Linked List** is a linear data structure where each node contains **three parts**:

1. **Data** – Stores the value.
2. **Prev** – A pointer to the previous node.
3. **Next** – A pointer to the next node.

This allows traversal of the list in **both forward and backward directions**, unlike singly linked lists.

Structure of a Node:

```
struct Node {
   int data;
   struct Node* prev;
   struct Node* next;
};
```

Basic Operations:

ADITYA UNIVERSITY

1. Create List

- Create nodes dynamically.
- Set prev and next pointers appropriately.
- Start with a head pointer pointing to the first node.

2. Insert Node

- **At Beginning**:
  - Create a new node.
  - Set its next to current head.
  - Set its prev to NULL.
  - Update the head's prev to point to the new node.
  - Make the new node the new head.
- **At End**:
  - Traverse to the last node.
  - Set its next to new node.
  - Set the new node's prev to last node and next to NULL.
- **At Specific Position**:
  - Traverse to the desired location.
  - Adjust prev and next pointers of adjacent nodes to insert the new node.

3. Delete Node

- **From Beginning**:

- Move head to the next node.
- Set new head's prev to NULL.
- Free the original head.

- **From End**:
  - Traverse to last node.
  - Set previous node's next to NULL.
  - Free the last node.
- **From Specific Position**:
  - Adjust the next of the previous node and prev of the next node.
  - Free the target node.

4. Display/Traverse

- **Forward Traversal**:
  Start from head and follow next pointers until NULL.
- **Backward Traversal**:
  Start from tail and follow prev pointers until NULL.

5. Search

- Traverse the list (either direction) and compare data of each node with the search value.

6. Count Nodes

- Traverse the list using either forward or backward pointers and maintain a counter.

ADITYA UNIVERSITY

Advantages:

- Bidirectional traversal.
- Easier to delete a node from any position (especially when a reference to the node is given).

Disadvantages:

- Requires extra memory for the prev pointer.
- Slightly more complex than singly linked list

PROGRAM:

```
#include<stdio.h>
#include<stdlib.h>
struct node
{
        int data;
        struct node *prev;
        struct node *next;
};
struct node *head;
void display()
{
```

```
        struct node *temp=head;
        while(temp!=NULL)
        {
                printf("%d-->%d\n",temp->data,temp->next);
                temp=temp->next;
        }
}
void create_list()
{
        struct node *n1,*n2,*n3,*n4;
        n1=(struct node *)malloc(sizeof(struct node));
        n2=(struct node *)malloc(sizeof(struct node));
        n3=(struct node *)malloc(sizeof(struct node));
        n4=(struct node *)malloc(sizeof(struct node));
        printf("enter 4 data values\n");
        scanf("%d%d%d%d",&n1->data,&n2->data,&n3->data,&n4->data);
        n1->prev=NULL;
        n1->next=n2;
        n2->prev=n1;
        n2->next=n3;
        n3->prev=n2;
        n3->next=n4;
        n4->prev=n3;
        n4->next=NULL;
        head=n1;
        display();
}
void insert_begin()
{
        struct node *newnode;
        newnode=(struct node *)malloc(sizeof(struct node));
        printf("enter new node data\n");
        scanf("%d",&newnode->data);
        newnode->data;
        newnode->next=head;
        newnode->prev=NULL;
        head=newnode;
        display();
}
void insert_end()
{
        struct node *newnode;
        newnode=(struct node *)malloc(sizeof(struct node));
        printf("enter new node data\n");
        scanf("%d",&newnode->data);
```

ADITYA UNIVERSITY

```
        newnode->data;
        newnode->next=NULL;
        struct node *temp=head;
        while(temp->next!=NULL)
        {
                temp=temp->next;
        }
        temp->next=newnode;
        newnode->prev=temp;
        newnode->next=NULL;
        display();
}
void insert_position()
{
        struct node *newnode;
        newnode=(struct node *)malloc(sizeof(struct node));
        printf("enter new node data\n");
        scanf("%d",&newnode->data);
        newnode->data;
        int i,position;
        printf("enter position\n");
        scanf("%d",&position);
        struct node *temp=head;
        for(i=1;i<position;i++)
        {
                if(temp->next!=NULL)
                {
                        temp=temp->next;
                }
        }
        newnode->next=temp->next;
        temp->next=newnode;
        newnode->prev=temp;
        display();
}
void deletion_begin()
{
        if(head==NULL)
        return;
        struct node *temp=head;
        head=head->next;
        free(temp);
        head->prev=NULL;
        display();
}
```

ADITYA UNIVERSITY

```
void deletion_end()
{
        if(head==NULL)
        return;
        if(head->next==NULL)
        {
                free(head);
                head=NULL;
                return;
        }
        struct node *temp=head;
        while(temp->next!=NULL&&temp->next->next!=NULL)
        {
                temp=temp->next;
        }
        struct node *lastnode=temp->next;
        temp->next=NULL;
        free(lastnode);
        display();
}
void deletion_position()
{
        struct node *temp,*nextnode;
        int position,i=1;
        printf("enter position\n");
        scanf("%d",&position);
        temp=head;
        while(i<position-1)
        {
                temp=temp->next;
                i=i+1;
        }
        nextnode=temp->next;
        nextnode->prev->next=nextnode->next;
        nextnode->next->prev=nextnode->prev;
        free(nextnode);
        display();
}
int main()
{
        int choice;
        while(1)
        {
```

ADITYA UNIVERSITY

```
        printf("\n1.create
list\n2.insert_at_beginning\n3.insert_at_ending\n4.insert_at_position\n5.deletion_at_beginning\n6.de
letion_at_ending\n7.deletion_at_position\n8.display\n9.exit\n");
        printf("enter choice:\n");
        scanf("%d",&choice);
        switch(choice)
        {
                case 1:create_list();
                    break;
                case 2:insert_begin();
                    break;
                case 3:insert_end();
                   break;
               case 4:insert_position();
                   break;
                case 5:deletion_begin();
                    break;
                case 6:deletion_end();
                    break;
                case 7:deletion_position();
                   break;
                case 8:display();
                   break;
                case 9:exit(0);
                   break;
           default:
                printf("Invalid choice!");
        }
   }
        return 0;
}
```

OUTPUT:

1.create list
2.insert_at_beginning
3.insert_at_ending
4.insert_at_position
5.deletion_at_beginning
6.deletion_at_ending
7.deletion_at_position
8.display
9.exit

```
enter choice:
1
enter 4 data values
7 8 9 4
7-->12807136
8-->12807168
9-->12807200
4-->0
1.create list
2.insert_at_beginning
3.insert_at_ending
4.insert_at_position
5.deletion_at_beginning
6.deletion_at_ending
7.deletion_at_position
8.display
9.exit

enter choice:
2
enter new node data
5
5-->12807104
7-->12807136
8-->12807168
9-->12807200
4-->0
1.create list
2.insert_at_beginning
3.insert_at_ending
4.insert_at_position
5.deletion_at_beginning
6.deletion_at_ending
7.deletion_at_position
8.display
9.exit

enter choice:
3
enter new node data
1
5-->12807104
7-->12807136
8-->12807168
9-->12807200
```

ADITYA UNIVERSITY

```
4-->12807264
1-->0
1.create list
2.insert_at_beginning
3.insert_at_ending
4.insert_at_position
5.deletion_at_beginning
6.deletion_at_ending
7.deletion_at_position
8.display
9.exit
enter choice:
4
enter new node data
2
enter position
2
5-->12807104
7-->12807296
2-->12807136
8-->12807168
9-->12807200
4-->12807264
1-->0
1.create list
2.insert_at_beginning
3.insert_at_ending
4.insert_at_position
5.deletion_at_beginning
6.deletion_at_ending
7.deletion_at_position
8.display
9.exit
enter choice:
5
7-->12807296
2-->12807136
8-->12807168
9-->12807200
4-->12807264
1-->0
1.create list
2.insert_at_beginning
3.insert_at_ending
4.insert_at_position
```

ADITYA UNIVERSITY

```
5.deletion_at_beginning
6.deletion_at_ending
7.deletion_at_position
8.display
9.exit
enter choice:
6
7-->12807296
2-->12807136
8-->12807168
9-->12807200
4-->0
1.create list
2.insert_at_beginning
3.insert_at_ending
4.insert_at_position
5.deletion_at_beginning
6.deletion_at_ending
7.deletion_at_position
8.display
9.exit
enter choice:
7
enter position
2
7-->12807136
8-->12807168
9-->12807200
4-->0
1.create list
2.insert_at_beginning
3.insert_at_ending
4.insert_at_position
5.deletion_at_beginning
6.deletion_at_ending
7.deletion_at_position
8.display
9.exit
enter choice:
8
7-->12807136
8-->12807168
9-->12807200
4-->0
1.create list
```

ADITYA UNIVERSITY

2.insert_at_beginning
3.insert_at_ending
4.insert_at_position
5.deletion_at_beginning
6.deletion_at_ending
7.deletion_at_position
8.display
9.exit
enter choice:
9

ADITYA UNIVERSITY

Practice 3.4

AIM: Write a C Program to implement Circular Linked List

DESCRIPTION:

A **Circular Linked List** is a linear data structure in which the last node is connected back to the first node, forming a circular chain. Unlike a standard linked list that terminates at the last node, a circular linked list allows continuous traversal through the list from any point.

Structure:

In a circular linked list, each node contains:

- **Data**: The value or information stored in the node.
- **Pointer**: A reference to the next node in the sequence.

The final node in the list does not point to NULL, but instead points back to the head (first node), forming a closed loop.

Types:

1. **Singly Circular Linked List** – Each node links to the next node; the last node points to the head.
2. **Doubly Circular Linked List** – Each node links to both the next and previous nodes; the last node's next points to the head, and the head's previous points to the last node.

ADITYA UNIVERSITY

Key Operations:

- **Insertion** – Can be performed at the beginning, end, or any specific position.
- **Deletion** – Can be performed from any position in the list.
- **Traversal** – Must use a loop that terminates when the starting node is encountered again.

Characteristics:

- No null reference is used for the end of the list.
- The list can be navigated from any node in a loop.
- It provides efficient performance in scenarios where looping over data is required.

Use Cases:

- Task scheduling (e.g., round-robin algorithm)
- Buffer management
- Applications that require continuous navigation through data

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
struct Node
{
   int data;
   struct Node* next;
};
struct Node* head = NULL;
void createNode()
{
   struct Node *n1, *n2, *n3;
   n1 = (struct Node*)malloc(sizeof(struct Node));
   n2 = (struct Node*)malloc(sizeof(struct Node));
   n3 = (struct Node*)malloc(sizeof(struct Node));
   printf("Enter 3 values:\n");
   scanf("%d %d %d", &n1->data, &n2->data, &n3->data);
   n1->next = n2;
   n2->next = n3;
   n3->next = n1;
   head = n1;
   display();
}
void display()
{
         struct Node* temp = head;
   do
         {
      printf("%d -> %d\n",temp->data,temp->next);
      temp = temp->next;
   } while (temp != head);
   printf("(head)\n");
}
void insertAtBeginning(int data)
{
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = data;
   if (head == NULL)
         {
      newNode->next = newNode;
      head = newNode;
   }
         else
         {
      struct Node* temp = head;
```

ADITYA UNIVERSITY

```
        while (temp->next != head)
                {
            temp = temp->next;
        }
        temp->next = newNode;
        newNode->next = head;
        head = newNode;
    }
    display();
}
void insertAtEnd(int data)
{
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    if (head == NULL)
            {
        newNode->next = newNode;
        head = newNode;
    }
            else
            {
        struct Node* temp = head;
        while (temp->next != head)
                {
            temp = temp->next;
        }
        temp->next = newNode;
        newNode->next = head;
    }
    display();
}
void insertAtPosition(int data, int position)
{
            int i;
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    if (position == 1)
            {
        insertAtBeginning(data);
        return;
    }
    struct Node* temp = head;
    for(i = 1; i < position - 1 && temp->next != head; i++)
            {
        temp = temp->next;
```

ADITYA UNIVERSITY

```
    }
    if (temp->next == head)
            {
        insertAtEnd(data);
    }
            else
            {
        newNode->next = temp->next;
        temp->next = newNode;
    }
    display();
}
void deleteAtBeginning()
{
    if (head == NULL)
            {
        printf("The list is empty.\n");
        return;
    }
    if (head->next == head)
            {
        free(head);
        head = NULL;
    }
            else
            {
        struct Node* temp = head;
        while (temp->next != head)
                    {
            temp = temp->next;
        }
        struct Node* toDelete = head;
        temp->next = head->next;
        head = head->next;
        free(toDelete);
    }
    display();
}
void deleteAtEnd()
{
    if (head == NULL)
            {
        printf("The list is empty.\n");
        return;
    }
```

ADITYA UNIVERSITY

```
    if (head->next == head)
            {
        free(head);
        head = NULL;
    }
            else
            {
        struct Node* temp = head;
        while (temp->next->next != head)
                    {
            temp = temp->next;
        }
        struct Node* toDelete = temp->next;
        temp->next = head;
        free(toDelete);
    }
    display();
}
void deleteAtPosition(int position)
{
            int i;
    if (head == NULL)
            {
        printf("The list is empty.\n");
        return;
    }
    if (position == 1)
            {
        deleteAtBeginning();
        return;
    }
    struct Node* temp = head;
    for (i = 1; i < position - 1 && temp->next != head; i++)
            {
        temp = temp->next;
    }
    if (temp->next == head)
            {
        printf("Invalid position!\n");
        return;
    }
    struct Node* toDelete = temp->next;
    temp->next = toDelete->next;
    free(toDelete);
    display();
```

ADITYA UNIVERSITY

```
}
int main()
{
   int choice, value, position;
   while (1)
         {

printf("\n1.create_list\n2.insert_at_beginning\n3.insert_at_ending\n4.insert_at_position\n5.diaplay\n
6.deletion_at_begining\n7.deletion_at_ending\n8.deletion_at_position\n9.exit\n");
      printf("Enter your choice: ");
      scanf("%d", &choice);
      switch (choice)
                  {
                        case 1:
                           createNode();
                              break;
         case 2:
            printf("Enter the value to insert at the beginning: ");
            scanf("%d", &value);
            insertAtBeginning(value);
            break;
         case 3:
            printf("Enter the value to insert at the end: ");
            scanf("%d", &value);
            insertAtEnd(value);
            break;
         case 4:
            printf("Enter the value to insert: ");
            scanf("%d", &value);
            printf("Enter the position to insert: ");
            scanf("%d", &position);
            insertAtPosition(value, position);
            break;
         case 5:
            display();
            break;
         case 6:
            deleteAtBeginning();
            break;
         case 7:
            deleteAtEnd();
            break;
         case 8:
            printf("Enter the position to delete: ");
            scanf("%d", &position);
```

ADITYA UNIVERSITY

```
            deleteAtPosition(position);
            break;
        case 9:
            printf("Exiting...\n");
            exit(0);
            break;
        default:
            printf("Invalid choice! Please try again.\n");
    }
  }
  return 0;
}
```

OUTPUT:

1.create_list

2.insert_at_beginning

3.insert_at_ending

4.insert_at_position

5.diaplay

6.deletion_at_begining

7.deletion_at_ending

8.deletion_at_position

9.exit

Enter your choice: 1

Enter 3 values:

4 5 6

4 -> 11824096

5 -> 11824128

6 -> 11824064

(head)

ADITYA UNIVERSITY

1.create_list

2.insert_at_beginning

3.insert_at_ending

4.insert_at_position

5.diaplay

6.deletion_at_begining

7.deletion_at_ending

8.deletion_at_position

9.exit

Enter your choice: 2

Enter the value to insert at the beginning: 4

4 -> 11824064

4 -> 11824096

5 -> 11824128

6 -> 11824160

(head)

ADITYA UNIVERSITY

1.create_list

2.insert_at_beginning

3.insert_at_ending

4.insert_at_position

5.diaplay

6.deletion_at_begining

7.deletion_at_ending

8.deletion_at_position

9.exit

Enter your choice: 3

Enter the value to insert at the end: 5

4 -> 11824064

4 -> 11824096

5 -> 11824128

6 -> 11824192

5 -> 11824160

(head)

1.create_list

2.insert_at_beginning

3.insert_at_ending

4.insert_at_position

5.diaplay

6.deletion_at_begining

7.deletion_at_ending

8.deletion_at_position

ADITYA UNIVERSITY

9.exit

Enter your choice: 4

Enter the value to insert: 6

Enter the position to insert: 2

4 -> 11824224

6 -> 11824064

4 -> 11824096

5 -> 11824128

6 -> 11824192

5 -> 11824160

(head)

1.create_list

2.insert_at_beginning

3.insert_at_ending

4.insert_at_position

5.diaplay

6.deletion_at_begining

7.deletion_at_ending

8.deletion_at_position

9.exit

Enter your choice: 6

6 -> 11824064

4 -> 11824096

5 -> 11824128

6 -> 11824192

5 -> 11824224

(head)

ADITYA UNIVERSITY

1.create_list

2.insert_at_beginning

3.insert_at_ending

4.insert_at_position

5.diaplay

6.deletion_at_begining

7.deletion_at_ending

8.deletion_at_position

9.exit

Enter your choice: 7

6 -> 11824064

4 -> 11824096

5 -> 11824128

6 -> 11824224

(head)

1.create_list

2.insert_at_beginning

3.insert_at_ending

4.insert_at_position

5.diaplay

6.deletion_at_begining

7.deletion_at_ending

8.deletion_at_position

9.exit

Enter your choice: 8

Enter the position to delete: 2

6 -> 11824096

5 -> 11824128

ADITYA UNIVERSITY

6 -> 11824224

(head)

1.create_list

2.insert_at_beginning

3.insert_at_ending

4.insert_at_position

5.diaplay

6.deletion_at_begining

7.deletion_at_ending

8.deletion_at_position

9.exit

Enter your choice: 9

Exiting...

Practice 4.1.1

AIM: Write a C Program to implement Stack and its operations using arrays

DESCRIPTION:

A **Stack** is a linear data structure that follows the **Last In First Out (LIFO)** principle. This means that the element inserted last is the one to be removed first. It works like a stack of plates — you can only remove the top one.

Stack Implementation Using Arrays:

A stack can be implemented using an array with a fixed size. A variable called top is used to track the index of the topmost element in the stack.

Basic Operations:

1. **Push**:
   Adds an element to the top of the stack.
   - If the stack is full (top == size - 1), it causes **Stack Overflow**.
2. **Pop**:
   Removes the topmost element from the stack.
   - If the stack is empty (top == -1), it causes **Stack Underflow**.
3. **Peek (or Top)**:
   Returns the value at the top of the stack without removing it.
4. **Display**:
   Displays all the elements in the stack from top to bottom

ADITYA UNIVERSITY

Structure:

```
#define MAX 100

int stack[MAX];

int top = -1;
```

Advantages:

- Simple and fast implementation.
- Efficient for problems like expression evaluation, backtracking, undo features, etc.

Applications of Stack:

- Expression evaluation (Infix, Prefix, Postfix)
- Function call management in recursion
- Undo operations in editors
- Parenthesis checking in compilers

PROGRAM:

```
#include <stdio.h>
#define MAX 5

int stack[MAX], top = -1;

void push() {
    if (top == MAX - 1) {
        printf("Stack Overflow\n");
    } else {
        int value;
        printf("Enter value to push: ");
        scanf("%d", &value);
        stack[++top] = value;
        printf("%d pushed to stack.\n", value);
    }
}

void pop() {
    if (top == -1) {
        printf("Stack Underflow\n");
    } else {
        printf("%d popped from stack.\n", stack[top--]);
    }
}

void peek() {
    if (top == -1) {
        printf("Stack is empty.\n");
    } else {
        printf("Top element: %d\n", stack[top]);
    }
}

void display() {
    if (top == -1) {
        printf("Stack is empty.\n");
    } else {
        printf("Stack elements: ");
        for (int i = 0; i <= top; i++) {
            printf("%d ", stack[i]);
        }
        printf("\n");
    }
}
```

ADITYA UNIVERSITY

```
int main()
{
   int choice;
   while (1) {
      printf("\nStack Operations:\n");
      printf("1. Push\n2. Pop\n3. Peek\n4. Display\n5. Exit\n");
      printf("Enter your choice: ");
      scanf("%d", &choice);
      switch (choice) {
         case 1: push(); break;
         case 2: pop(); break;
         case 3: peek(); break;
         case 4: display(); break;
         case 5:
            printf("Exiting...\n");
            return 0;
         default:
            printf("Invalid choice! Try again.\n");
      }
   }
}
```

ADITYA UNIVERSITY

OUTPUT:

```
Stack Operations:
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter value to push: 10
10 pushed to stack.

Stack Operations:
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter value to push: 20
20 pushed to stack.

Stack Operations:
```

```
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 3
Top element: 20

Stack Operations:
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack elements: 10 20

Stack Operations:
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 2
20 popped from stack.

Stack Operations:
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack elements: 10

Stack Operations:
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack elements: 10
```

ADITYA UNIVERSITY

Practice 4.1.2

AIM: Write a C Program to implement Stack and its operations using Linked List

DESCRIPTION:

A **Stack** is a linear data structure that follows the **Last In, First Out (LIFO)** principle. In this structure, the element inserted last is the one removed first. Unlike array-based stacks, stacks using linked lists are **dynamic** in size and do not suffer from overflow unless system memory is exhausted.

Why Use Linked List for Stack?

- Dynamic memory allocation — stack grows or shrinks as needed.
- No predefined size (unlike arrays).
- Efficient in terms of memory usage when the size of data is unpredictable.

Structure:

Each node in the linked list contains:

- **data**: the value to be stored.
- **next**: a pointer to the next node.

A pointer top is used to track the topmost element of the stack.

ADITYA UNIVERSITY

Basic Operations:

1. **Push**:
   - Creates a new node.
   - Inserts the new node at the beginning (top) of the list.
   - Updates top to point to the new node.
   - Time Complexity: **O(1)**
2. **Pop**:
   - Removes the node at the top.
   - Frees memory and updates top to the next node.
   - Handles underflow when stack is empty.
   - Time Complexity: **O(1)**
3. **Peek**:
   - Displays the top element without removing it.
   - Useful for checking the current element at the top.
4. **Display**:
   - Traverses the list from top to NULL, printing all elements.

Advantages:

- No memory wastage due to dynamic allocation.

- Overcomes size limitations of array-based stack.
- Efficient insertion and deletion at the beginning.

Applications:

- Function call handling (recursion).
- Expression evaluation (Postfix, Prefix).
- Backtracking algorithms (e.g., maze, puzzles).
- Undo operations in text editors or software.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node* next;
};

struct Node* top = NULL;

void push() {
   int value;
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   if (newNode == NULL) {
      printf("Stack Overflow\n");
      return;
   }
   printf("Enter value to push: ");
   scanf("%d", &value);
   newNode->data = value;
   newNode->next = top;
   top = newNode;
   printf("%d pushed to stack.\n", value);
}

void pop() {
   if (top == NULL) {
      printf("Stack Underflow! No element to pop.\n");
   } else {
      struct Node* temp = top;
      printf("%d popped from stack.\n", top->data);
      top = top->next;
      free(temp);
   }
}
```

ADITYA UNIVERSITY

```
void peek() {
    if (top == NULL) {
        printf("Stack is empty.\n");
    } else {
        printf("Top element: %d\n", top->data);
    }
}

void display() {
    if (top == NULL) {
        printf("Stack is empty.\n");
    } else {
        struct Node* temp = top;
        printf("Stack elements: ");
        while (temp != NULL) {
            printf("%d ", temp->data);
            temp = temp->next;
        }
        printf("\n");
    }
}

void freeStack() {
    struct Node* temp;
    while (top != NULL) {
        temp = top;
        top = top->next;
        free(temp);
    }
}

int main() {
    int choice;
    while (1) {
        printf("\nStack Operations:\n");
        printf("1. Push\n2. Pop\n3. Peek\n4. Display\n5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1: push(); break;
            case 2: pop(); break;
            case 3: peek(); break;
            case 4: display(); break;
            case 5:
                freeStack();
                printf("Exiting...\n");
                return 0;
            default:
```

ADITYA UNIVERSITY

```
            printf("Invalid choice! Try again.\n");
        }
    }
}
```

OUTPUT:

```
Stack Operations:
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter value to push: 10
10 pushed to stack.

Stack Operations:
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter value to push: 20
20 pushed to stack.

Stack Operations:
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 3
Top element: 20

Stack Operations:
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack elements: 20 10
```

ADITYA UNIVERSITY

```
Stack Operations:
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 2
20 popped from stack.

Stack Operations:
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack elements: 10

Stack Operations:
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice:
=== Session Ended. Please Run the code again ===
```

ADITYA UNIVERSITY

Practice 4.2

AIM: Write a C Program to Implement a program to evaluate a postfix expression

DESCRIPTION:

A **postfix expression** (also known as Reverse Polish Notation) is a mathematical notation in which **operators follow their operands**. It eliminates the need for parentheses to define operation precedence.

Example:

- Infix Expression: 3 + 4 * 2
- Postfix Expression: 3 4 2 * +
- Evaluation:
  → 4 * 2 = 8
  → 3 + 8 = 11

Key Points:

- Operands are directly pushed onto the stack.
- When an operator is encountered:
  - Pop the top **two elements**.
  - Apply the operation.
  - Push the result back onto the stack.
- Final result is the **only element left** in the stack.

Algorithm:

1. **Start**
2. Read the postfix expression (assume it is space-separated for simplicity).
3. Initialize an empty stack.
4. Scan the postfix expression from left to right.
5. For each token in the expression:
   - If it is an **operand**, convert to integer and push onto the stack.
   - If it is an **operator**, pop the top two elements, apply the operator, and push the result back.
6. After scanning all tokens, the result is at the top of the stack.
7. Print the result.
8. **End**

ADITYA UNIVERSITY

PROGRAM:

```
#include <stdio.h>
#include <ctype.h>
#include <stdlib.h>
#include <string.h>

#define MAX 100

int stack[MAX];
int top = -1;

void push(int value) {
   if (top == MAX - 1) {
      printf("Stack Overflow!\n");
```

```
        return;
    }
    stack[++top] = value;
}

int pop() {
    if (top == -1) {
        printf("Stack Underflow!\n");
        exit(1);
    }
    return stack[top--];
}

int evaluatePostfix(char* expr) {
    int i = 0;
    while (expr[i] != '\0') {
        if (expr[i] == ' ') {
            i++;
            continue;
        }

        // If operand (multi-digit supported)
        if (isdigit(expr[i])) {
            int num = 0;
            while (isdigit(expr[i])) {
                num = num * 10 + (expr[i] - '0');
                i++;
            }
            push(num);
        }
        // If operator
        else {
            int val2 = pop();
            int val1 = pop();
            switch (expr[i]) {
                case '+': push(val1 + val2); break;
                case '-': push(val1 - val2); break;
                case '*': push(val1 * val2); break;
                case '/': push(val1 / val2); break;
                default:
                    printf("Invalid operator encountered: %c\n", expr[i]);
                    exit(1);
            }
            i++;
        }
```

ADITYA UNIVERSITY

```
    }
    return pop();
}

int main()
{
    char expr[MAX];

    printf("Enter a postfix expression (space-separated): ");
    fgets(expr, MAX, stdin);

    int result = evaluatePostfix(expr);
    printf("Result of the expression: %d\n", result);

    return 0;
}
```

OUTPUT:

Enter a postfix expression (space-separated): 5 6 2 + * 12 4 / -

Result of the expression: 46

ADITYA UNIVERSITY

Practice 4.3.1

AIM: Write a C Program to implement Queue and its operations using arrays

DESCRIPTION:

A **Queue** is a linear data structure that follows the **First In, First Out (FIFO)** principle. The element that is inserted first is the one that gets removed first. Queues are widely used in various real-life and computer scenarios, such as task scheduling, print spooling, and CPU job handling.

Why Use Arrays for Queue?

- Simple and easy to implement.
- Efficient for fixed-size operations.
- Suitable for situations where the maximum number of elements is known.

Structure:

A queue is typically implemented using:

- An **array** to store elements.
- Two integer variables:
  - front – points to the first element.
  - rear – points to the last inserted element.

ADITYA UNIVERSITY

Basic Operations:

1. **Enqueue (Insertion)**:
   - Adds an element at the rear end.
   - Increment the rear index.
   - Check for overflow (if rear == MAX - 1).
   - Time Complexity: **O(1)**
2. **Dequeue (Deletion)**:
   - Removes the element from the front.
   - Increment the front index.
   - Check for underflow (if front > rear or queue is empty).
   - Time Complexity: **O(1)**
3. **Peek**:
   - Returns the element at the front of the queue without removing it.
4. **Display**:
   - Shows all elements from front to rear.

Limitations:

- Fixed size (defined by array size).
- Inefficient use of space after multiple deletions (overcome by using Circular Queue).

Applications:

- Printer queue management.
- CPU scheduling.
- Customer service simulations.
- Breadth-First Search (BFS) in graphs.

PROGRAM:

```
#include<stdio.h>
#include <stdlib.h>
#define SIZE 10

void enQueue(int);
void deQueue();
void display();

int queue[SIZE], front = -1, rear = -1;

void main()
{
   int value, choice;
   while(1){
      printf("\n\n***** MENU *****\n");
      printf("1. Insertion\n2. Deletion\n3. Display\n4. Exit");
      printf("\nEnter your choice: ");
      scanf("%d",&choice);
      switch(choice)
      {
         case 1:
            printf("Enter the value to be insert: ");
            scanf("%d",&value);
            enQueue(value);
            break;
         case 2:
            deQueue();
            break;
         case 3:
            display();
            break;
         case 4:
            exit(0);
         default:
            printf("\nWrong selection!!! Try again!!!");
      }
```

ADITYA UNIVERSITY

```
    }
}

void enQueue(int value)
{
    if(rear == SIZE-1)
        printf("\nQueue is Full");
    else
    {
        if(front == -1)
            front = 0;
        rear++;
        queue[rear] = value;
        printf("\nInsertion success!!!");
    }
}

void deQueue()
{
    if(front == -1 || front > rear)
        printf("\nQueue is Empty");
    else
    {
        printf("\nDeleted : %d", queue[front]);
        front++;
        if(front > rear)
            front = rear = -1;
    }
}

void display()
{
    if(front == -1)
        printf("\nQueue is Empty!!!");
    else
    {
        printf("\nQueue elements are:\n");
        for(int i=front; i<=rear; i++)
            printf("%d\t", queue[i]);
    }
}
```

ADITYA UNIVERSITY

OUTPUT:

```
***** MENU *****
1. Insertion
2. Deletion
3. Display
4. Exit
Enter your choice: 1
Enter the value to be insert: 10

Insertion success!!!
***** MENU *****
1. Insertion
2. Deletion
3. Display
4. Exit
Enter your choice: 1
Enter the value to be insert: 20

Insertion success!!!

***** MENU *****
1. Insertion
2. Deletion
3. Display
4. Exit
Enter your choice: 3

Queue elements are:
10      20

***** MENU *****
1. Insertion
2. Deletion
3. Display
4. Exit
Enter your choice: 2

Deleted : 10

***** MENU *****
1. Insertion
2. Deletion
3. Display
4. Exit
Enter your choice: 3

Queue elements are:
20
```

ADITYA UNIVERSITY

Practice 4.3.2

AIM: Write a C Program to implement Queue and its operations using Linked List

DESCRIPTION:

A **Queue** is a linear data structure that works on the **First In First Out (FIFO)** principle. In this structure, the element inserted first is removed first.

Using a **linked list** to implement a queue allows **dynamic memory allocation**. This eliminates the limitation of fixed size that exists in array-based queues.

Why Linked List for Queue?

- No need to define the size of the queue beforehand.
- Efficient memory utilization.
- Ideal for situations where the number of elements is unknown or constantly changing.

Structure:

Each node in the linked list contains:

- **data**: the value of the element.
- **next**: a pointer to the next node.

Two pointers are maintained:

- **front**: points to the first node (for deletion).
- **rear**: points to the last node (for insertion).

Operations:

ADITYA UNIVERSITY

1. **Enqueue (Insertion):**
   - Create a new node.
   - Insert it at the end of the list.
   - Update the rear pointer.
   - If the queue is empty, both front and rear point to the new node.
2. **Dequeue (Deletion):**
   - Remove the node pointed by front.
   - Update front to point to the next node.
   - If front becomes NULL, set rear to NULL as well (queue becomes empty).
3. **Display:**
   - Traverse the list from front to rear and print each node's data.

Advantages:

- Dynamic size – grows or shrinks at runtime.
- No overflow unless system memory is full.
- More flexible than array implementation.

Applications:

- Scheduling tasks (CPU/job scheduling).
- Print queues in operating systems.
- Buffer handling (data packets, keyboard buffer).
- Breadth-First Search in graph algorithms.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node* front = NULL;
struct Node* rear = NULL;

void enqueue() {
    int value;
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed! Cannot enqueue.\n");
        return;
    }
    printf("Enter the value to enqueue: ");
    scanf("%d", &value);
    newNode->data = value;
    newNode->next = NULL;

    if (rear == NULL) { // If queue is empty
        front = rear = newNode;
    } else {
        rear->next = newNode;
        rear = newNode;
    }
    printf("%d enqueued to the queue.\n", value);
}

void dequeue() {
    if (front == NULL) {
        printf("Queue Underflow! The queue is empty.\n");
        return;
    }
    struct Node* temp = front;
    printf("%d dequeued from the queue.\n", front->data);
    front = front->next;
    if (front == NULL) {
        rear = NULL;
    }
    free(temp);
```

ADITYA UNIVERSITY

```
}

void display() {
    if (front == NULL) {
        printf("Queue is empty.\n");
        return;
    }
    struct Node* temp = front;
    printf("Queue elements: ");
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

int main() {
    int choice;
    while (1) {
        printf("\nQueue Operations:\n");
        printf("1. Enqueue\n2. Dequeue\n3. Display\n4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1: enqueue(); break;
            case 2: dequeue(); break;
            case 3: display(); break;
            case 4: printf("Exiting program...\n"); exit(0);
            default: printf("Invalid choice! Try again.\n");
        }
    }
    return 0;
}
```

ADITYA UNIVERSITY

OUTPUT:

```
Queue Operations:
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter the value to enqueue: 10
10 enqueued to the queue.

Queue Operations:
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter the value to enqueue: 20
20 enqueued to the queue.

Queue Operations:
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 3
Queue elements: 10 20

Queue Operations:
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 2
10 dequeued from the queue.

Queue Operations:
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 3
Queue elements: 20
```

ADITYA UNIVERSITY

Practice 4.4

AIM: Write a c program to Implement Circular Queue using Arrays

DESCRIPTION:

A **Circular Queue** is a linear data structure that follows the **First In First Out (FIFO)** principle but connects the **last** position back to the **first**, forming a **circle**. It efficiently utilizes memory compared to a linear queue.

In a regular queue implemented using arrays, once the rear reaches the end, no new element can be inserted even if the front has moved ahead. This leads to **wastage of space**.

In **circular queues**, we use the following formula for index rotation:

- rear = (rear + 1) % SIZE
- front = (front + 1) % SIZE

This allows the queue to wrap around and reuse the empty spaces.

Operations:

1. **Enqueue:**
   - Add an element at the rear.
   - Check for overflow condition: (rear + 1) % SIZE == front.
2. **Dequeue:**
   - Remove an element from the front.
   - Check for underflow condition: front == -1.
3. **Display:**
   - Traverse from front to rear, wrapping around if necessary.

ADITYA UNIVERSITY

Advantages of Circular Queue:

- Efficient memory utilization.
- Eliminates the need for shifting elements.
- Useful in buffer management (e.g., keyboard buffers, streaming data).

Applications:

- CPU Scheduling (Round-Robin).
- IO Buffers.
- Network Data Packet Management.
- Traffic Light Management Systems.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>

#define SIZE 5

int cQueue[SIZE];
int front = -1, rear = -1;

// Function Prototypes
void enQueue(int value);
void deQueue();
void display();

int main() {
   int choice, value;

   while (1) {
      printf("\n====== Circular Queue Menu ======\n");
      printf("1. Enqueue (Insert)\n");
      printf("2. Dequeue (Delete)\n");
      printf("3. Display\n");
      printf("4. Exit\n");
      printf("Enter your choice: ");
      scanf("%d", &choice);

      switch (choice) {
      case 1:
         printf("Enter the value to insert: ");
         scanf("%d", &value);
         enQueue(value);
         break;
      case 2:
         deQueue();
         break;
      case 3:
         display();
         break;
      case 4:
         printf("Exiting program...\n");
         exit(0);
      default:
         printf("Invalid choice! Please try again.\n");
      }
   }
```

ADITYA UNIVERSITY

```
    return 0;
}

void enQueue(int value) {
    if ((front == 0 && rear == SIZE - 1) || (front == rear + 1)) {
        printf("Queue is Full! Insertion not possible.\n");
        return;
    }

    if (front == -1) {
        front = rear = 0;
    } else if (rear == SIZE - 1 && front != 0) {
        rear = 0;
    } else {
        rear++;
    }

    cQueue[rear] = value;
    printf("%d inserted into the queue.\n", value);
}

void deQueue() {
    if (front == -1) {
        printf("Queue is Empty! Deletion not possible.\n");
        return;
    }

    printf("%d deleted from the queue.\n", cQueue[front]);

    if (front == rear) {
        // Only one element was in the queue
        front = rear = -1;
    } else if (front == SIZE - 1) {
        front = 0;
    } else {
        front++;
    }
}

void display() {
    if (front == -1) {
        printf("Queue is Empty!\n");
        return;
    }
```

ADITYA UNIVERSITY

```
    printf("Queue elements: ");
    int i = front;
    if (front <= rear) {
        while (i <= rear) {
            printf("%d ", cQueue[i]);
            i++;
        }
    } else {
        while (i < SIZE) {
            printf("%d ", cQueue[i]);
            i++;
        }
        i = 0;
        while (i <= rear) {
            printf("%d ", cQueue[i]);
            i++;
        }
    }
    printf("\n");
}
```

ADITYA UNIVERSITY

OUTPUT:

```
====== Circular Queue Menu ======
1. Enqueue (Insert)
2. Dequeue (Delete)
3. Display
4. Exit
Enter your choice: 1
Enter the value to insert: 10
10 inserted into the queue.

====== Circular Queue Menu ======
1. Enqueue (Insert)
2. Dequeue (Delete)
3. Display
4. Exit
Enter your choice: 1
Enter the value to insert: 20
20 inserted into the queue.

====== Circular Queue Menu ======
1. Enqueue (Insert)
2. Dequeue (Delete)
3. Display
```

```
4. Exit
Enter your choice: 3
Queue elements: 10 20

====== Circular Queue Menu ======
1. Enqueue (Insert)
2. Dequeue (Delete)
3. Display
4. Exit
Enter your choice: 2
10 deleted from the queue.

====== Circular Queue Menu ======
1. Enqueue (Insert)
2. Dequeue (Delete)
3. Display
4. Exit
Enter your choice: 3
Queue elements: 20

====== Circular Queue Menu ======
1. Enqueue (Insert)
2. Dequeue (Delete)
3. Display
4. Exit
Enter your choice:
```

ADITYA UNIVERSITY

Practice 5.1

AIM: Write a C to Implement Binary search tree (BST).

DESCRIPTION:

A **Binary Search Tree (BST)** is a binary tree in which:

- The **left child** of a node contains only nodes with **values less than the node's value**.
- The **right child** contains only nodes with **values greater than the node's value**.
- This property holds recursively for all nodes.

Applications:

- Efficient searching, insertion, and deletion (average time complexity: O(log n))
- Used in databases, file systems, and searching algorithms

Operations to Implement:

- **Insertion**
  - Recursively place the new value in the left or right subtree based on comparison.
- **Deletion**
  - Three cases:
    - Node is a leaf (no children): delete directly.
    - Node has one child: replace node with the child.
    - Node has two children: replace with inorder successor (minimum in right subtree).
- **In-order Traversal**
  - Left subtree → Root → Right subtree (prints values in sorted order).

ADITYA UNIVERSITY

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>

// Define node structure
struct Node {
   int data;
   struct Node* left;
   struct Node* right;
};

// Create a new node
struct Node* createNode(int value) {
   struct Node* newNode = (struct Node*) malloc(sizeof(struct Node));
```

```
    newNode->data = value;
    newNode->left = newNode->right = NULL;
    return newNode;
}

// Insert a node into BST
struct Node* insert(struct Node* root, int value) {
    if (root == NULL) {
        return createNode(value);
    }

    if (value < root->data) {
        root->left = insert(root->left, value);
    } else if (value > root->data) {
        root->right = insert(root->right, value);
    } else {
        printf("Duplicate value %d not inserted.\n", value);
    }

    return root;
}

// Find the minimum value node in the tree
struct Node* findMin(struct Node* root) {
    while (root && root->left != NULL) {
        root = root->left;
    }
    return root;
}

// Delete a node from BST
struct Node* delete(struct Node* root, int value) {
    if (root == NULL) {
        printf("Value %d not found in the tree.\n", value);
        return root;
    }

    if (value < root->data) {
        root->left = delete(root->left, value);
    } else if (value > root->data) {
        root->right = delete(root->right, value);
    } else {
        // Node to be deleted found
        if (root->left == NULL) {
            struct Node* temp = root->right;
```

ADITYA UNIVERSITY

```
            free(root);
            return temp;
        } else if (root->right == NULL) {
            struct Node* temp = root->left;
            free(root);
            return temp;
        }

        // Node with two children
        struct Node* temp = findMin(root->right);
        root->data = temp->data;
        root->right = delete(root->right, temp->data);
    }
    return root;
}

// In-order traversal (Left, Root, Right)
void inOrder(struct Node* root) {
    if (root != NULL) {
        inOrder(root->left);
        printf("%d ", root->data);
        inOrder(root->right);
    }
}

// Main function
int main() {
    struct Node* root = NULL;
    int choice, value;

    while (1) {
        printf("\n--- BST Menu ---\n");
        printf("1. Insert\n2. Delete\n3. In-order Traversal\n4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
        case 1:
            printf("Enter value to insert: ");
            scanf("%d", &value);
            root = insert(root, value);
            break;
        case 2:
            printf("Enter value to delete: ");
            scanf("%d", &value);
```

ADITYA UNIVERSITY

```
            root = delete(root, value);
            break;
        case 3:
            printf("In-order Traversal: ");
            inOrder(root);
            printf("\n");
            break;
        case 4:
            exit(0);
        default:
            printf("Invalid choice. Try again.\n");
        }
    }

    return 0;
}
```

OUTPUT:

```
--- BST Menu ---
1. Insert
2. Delete
3. In-order Traversal
4. Exit
Enter your choice: 1
Enter value to insert: 10

--- BST Menu ---
1. Insert
2. Delete
3. In-order Traversal
4. Exit
Enter your choice: 1
Enter value to insert: 20

--- BST Menu ---
1. Insert
2. Delete
3. In-order Traversal
4. Exit
Enter your choice: 3
In-order Traversal: 10 20

--- BST Menu ---
1. Insert
2. Delete
```

ADITYA UNIVERSITY

3. In-order Traversal
4. Exit
Enter your choice: 2
Enter value to delete: 10

--- BST Menu ---
1. Insert
2. Delete
3. In-order Traversal
4. Exit
Enter your choice: 3
In-order Traversal: 20

--- BST Menu ---
1. Insert
2. Delete
3. In-order Traversal
4. Exit
Enter your choice:
=== Session Ended. Please Run the code again ===

ADITYA UNIVERSITY

Practice 5.2

AIM: Write a C Program to Implement Binary search Tree (BST) Traversals

DESCRIPTION:

A **Binary Search Tree (BST)** is a binary tree where for every node:

- The value of all nodes in the left subtree is **less than** the node's value.
- The value of all nodes in the right subtree is **greater than** the node's value.

Tree traversal refers to the process of visiting each node in the tree exactly once in a specific order. In BSTs, traversal is used for tasks like displaying, searching, or modifying node values

Types of Tree Traversals:

. In-order Traversal (Left → Root → Right)

- Visits the left subtree, then the root node, then the right subtree.
- In a BST, this traversal gives the nodes in **sorted (ascending) order**.
- **Use Case:** Displaying BST in sorted order.

2. Pre-order Traversal (Root → Left → Right)

- Visits the root node first, then the left subtree, then the right subtree.
- **Use Case:** Used to **create a copy** of the tree or to **generate prefix** expressions in expression trees.

3. Post-order Traversal (Left → Right → Root)

- Visits the left subtree, right subtree, and then the root node.
- **Use Case:** Used to **delete** the tree or **evaluate postfix** expressions.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>

// Node structure
struct Node {
   int data;
   struct Node* left;
   struct Node* right;
};

// Create a new node
struct Node* createNode(int data) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = data;
   newNode->left = newNode->right = NULL;
   return newNode;
}

// Insert into BST
struct Node* insert(struct Node* root, int data) {
   if (root == NULL)
      return createNode(data);

   if (data < root->data)
```

ADITYA UNIVERSITY

```
        root->left = insert(root->left, data);
    else if (data > root->data)
        root->right = insert(root->right, data);

    return root;
}

// In-order traversal: Left -> Root -> Right
void inOrder(struct Node* root) {
    if (root != NULL) {
        inOrder(root->left);
        printf("%d ", root->data);
        inOrder(root->right);
    }
}

// Pre-order traversal: Root -> Left -> Right
void preOrder(struct Node* root) {
    if (root != NULL) {
        printf("%d ", root->data);
        preOrder(root->left);
        preOrder(root->right);
    }
}

// Post-order traversal: Left -> Right -> Root
void postOrder(struct Node* root) {
    if (root != NULL) {
        postOrder(root->left);
        postOrder(root->right);
        printf("%d ", root->data);
    }
}

int main() {
    struct Node* root = NULL;
    int choice, value;

    // Menu-driven interface
    while (1) {
        printf("\n==== Binary Search Tree Menu ====\n");
        printf("1. Insert\n2. In-order Traversal\n3. Pre-order Traversal\n4. Post-order Traversal\n5.
Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
```

ADITYA UNIVERSITY

```
    switch(choice) {
        case 1:
            printf("Enter value to insert: ");
            scanf("%d", &value);
            root = insert(root, value);
            break;
        case 2:
            printf("In-order Traversal: ");
            inOrder(root);
            printf("\n");
            break;
        case 3:
            printf("Pre-order Traversal: ");
            preOrder(root);
            printf("\n");
            break;
        case 4:
            printf("Post-order Traversal: ");
            postOrder(root);
            printf("\n");
            break;
        case 5:
            printf("Exiting program.\n");
            exit(0);
        default:
            printf("Invalid choice! Try again.\n");
    }
  }

  return 0;
}
```

OUTPUT:

```
==== Binary Search Tree Menu ====
1. Insert
2. In-order Traversal
3. Pre-order Traversal
4. Post-order Traversal
5. Exit
Enter your choice: 1
Enter value to insert: 40
Enter your choice: 1
Enter value to insert: 20
```

ADITYA UNIVERSITY

```
Enter your choice: 1
Enter value to insert: 60
Enter your choice: 1
Enter value to insert: 10
Enter your choice: 1
Enter value to insert: 30
Enter your choice: 2
In-order Traversal: 10 20 30 40 60
Enter your choice: 3
Pre-order Traversal: 40 20 10 30 60
Enter your choice: 4
Post-order Traversal: 10 30 20 60 40
```

ADITYA UNIVERSITY

Practice 5.3

AIM: Write a C Program to implement Graph Traversal using Breadth First Search (BFS)

DESCRIPTION:

A **graph** is a non-linear data structure consisting of a set of **vertices (nodes)** connected by **edges**. Graphs can be classified as **directed** (where edges have a direction) or **undirected** (where edges do not have a direction).

Graph Traversal refers to the process of visiting, checking, and/or updating each node in the graph. In traversal, we can use several strategies, and **Breadth First Search (BFS)** is one of the most common techniques.

Breadth First Search (BFS) is an algorithm used to traverse or search through a graph level by level. Starting at the **source node**, BFS explores all its neighbors before moving to their neighbors.

- **Queue** is used to implement BFS traversal as it works in a **FIFO (First In, First Out)** manner.
- It systematically visits each node level by level.
- BFS is particularly useful for finding the **shortest path** in **unweighted graphs**.

▯ BFS Characteristics:

1. **Level-Order Traversal:** It explores all the nodes at the present depth level before moving on to nodes at the next depth level.
2. **Uses a Queue:** BFS uses a queue data structure to keep track of the nodes to visit next.
3. **Connected Graphs:** BFS can be used to determine if a graph is connected (i.e., all nodes can be reached from any starting node).
4. **Shortest Path:** BFS is ideal for finding the shortest path in an unweighted graph.

ADITYA UNIVERSITY

▯ Algorithm: Breadth First Search (BFS)

1. **Initialize**:
   - A **queue** to hold the nodes to visit.
   - A **visited array** to track the visited nodes (to avoid cycles and redundant visits).
2. **Start**:
   - Enqueue the **starting node** and mark it as visited.
3. **While the queue is not empty**:
   - **Dequeue** the front element from the queue (current node).
   - Visit and process the current node (e.g., print its value).
   - **Enqueue** all unvisited adjacent nodes of the current node, and mark them as visited.
4. **Repeat** the process until the queue becomes empty.

▯ Use Cases of BFS:

- **Shortest Path in Unweighted Graphs:** BFS guarantees that when a node is visited, the shortest path from the source to that node is found.
- **Web Crawlers:** In web crawlers, BFS can be used to explore web pages level by level, starting from a base URL.
- **Peer-to-Peer Networks:** BFS can be used to find the **shortest path** between peers in networks like BitTorrent.
- **Social Networks:** BFS can be used to determine the **degree of separation** between two users in a social network.
- **Finding Connected Components:** BFS can help identify the connected components in a graph (groups of nodes where any two nodes are connected by edges).

⍰ Graph Representation for BFS:

BFS can be implemented on **graphs represented by adjacency lists** or **adjacency matrices**. Here's an example using an **adjacency list**:

Graph Example:

Graph (Adjacency List):

0 -> [1, 2]
1 -> [0, 3]
2 -> [0, 4]
3 -> [1]
4 -> [2]

Steps of BFS with Example:

Let's take the above graph and start BFS from node 0.

1. **Start at node 0**:
   - Visit node 0 → Queue: [0] → Visited: [0]
2. **Visit the neighbors of node 0**:
   - Node 1 and Node 2 are neighbors of node 0.
   - Add them to the queue and mark them visited → Queue: [1, 2] → Visited: [0, 1, 2]
3. **Dequeue node 1**:
   - Visit node 1 → Queue: [2] → Visited: [0, 1, 2]
   - Node 3 is a neighbor of node 1, add it to the queue → Queue: [2, 3] → Visited: [0, 1, 2, 3]
4. **Dequeue node 2**:
   - Visit node 2 → Queue: [3] → Visited: [0, 1, 2, 3]
   - Node 4 is a neighbor of node 2, add it to the queue → Queue: [3, 4] → Visited: [0, 1, 2, 3, 4]
5. **Dequeue node 3**:
   - Visit node 3 → Queue: [4] → Visited: [0, 1, 2, 3, 4]
   - Node 3 has no new neighbors.
6. **Dequeue node 4**:
   - Visit node 4 → Queue: [] → Visited: [0, 1, 2, 3, 4]
   - Node 4 has no new neighbors.
7. **End**: The queue is empty, and BFS traversal is complete. The nodes were visited in the order: **0 → 1 → 2 → 3 → 4**.

ADITYA UNIVERSITY

PROGRAM:

```
#include <stdio.h>

int n, i, j, visited[10], queue[10], front = -1, rear = -1;
int adj[10][10];

void bfs(int v)
{
    for (i = 1; i <= n; i++)
        if (adj[v][i] && !visited[i])
            queue[++rear] = i;
    if (front <= rear)
    {
        visited[queue[front]] = 1;
        bfs(queue[front++]);
    }
}

 int main()
{
    int v;
    printf("Enter the number of vertices: ");
    scanf("%d", &n);
    for (i = 1; i <= n; i++)
    {
        queue[i] = 0;
        visited[i] = 0;
    }
    printf("Enter graph data in matrix form:    \n");
    for (i = 1; i <= n; i++)
        for (j = 1; j <= n; j++)
            scanf("%d", &adj[i][j]);
    printf("Enter the starting vertex: ");
    scanf("%d", &v);
    bfs(v);
    printf("The node which are reachable are:    \n");
    for (i = 1; i <= n; i++)
        if (visited[i])
            printf("%d\t", i);
        else
            printf("BFS is not possible. Not all nodes are reachable");
    return 0;
}
```

ADITYA UNIVERSITY

OUTPUT:

```
Enter the number of vertices: 4
Enter graph data in matrix form:
0 1 1 0
1 0 0 1
1 0 0 1
0 1 1 0
Enter the starting vertex: 2
The node which are reachable are:
1        2        3        4

=== Code Execution Successful ===
```

ADITYA UNIVERSITY

Practice 5.4

AIM: Write a C Program to implement Graph Traversal using Depth First Search (DFS)

DESCRIPTION:

Depth First Search (DFS) is a fundamental algorithm used to traverse or search through graphs and trees. The algorithm starts at a selected node (known as the source) and explores as far as possible along each branch before backtracking.

DFS uses the concept of **stack** (either explicit or via recursion) to remember the path taken and backtrack when necessary.

Graph Representation

Graphs can be represented using:

1. **Adjacency Matrix** – A 2D array where matrix[i][j] = 1 if there is an edge between node i and j.
2. **Adjacency List** – A list where each node stores its neighbors.

DFS Working Steps:

1. Start from a given node.
2. Visit the node and mark it as visited.
3. Recursively visit all unvisited adjacent nodes.
4. Backtrack when no unvisited adjacent node remains

ADITYA UNIVERSITY

DFS Characteristics:

Feature	Details
Data Structure Used	Stack or recursion
Time Complexity	O(V + E) where V = vertices, E = edges
Space Complexity	O(V) due to recursion stack or visited[]
Traversal Type	Depth-based (go as far as possible)
Applicable Graphs	Directed or Undirected, Connected/Disconnected
Applications	Cycle detection, Topological sorting, Puzzle solving, Path finding

PROGRAM:

```
#include <stdio.h>
#define MAX 10
int adjMatrix[MAX][MAX];
int visited[MAX];
int n; // Number of vertices

// DFS Function
void DFS(int vertex) {
    printf("Visited %d\n", vertex);
    visited[vertex] = 1;

    for (int i = 0; i < n; i++) {
        if (adjMatrix[vertex][i] == 1 && !visited[i]) {
            DFS(i);
        }
    }
}

int main() {
    int edges, src, dest;

    printf("Enter the number of vertices: ");
    scanf("%d", &n);

    // Initialize adjacency matrix and visited array
    for (int i = 0; i < n; i++) {
        visited[i] = 0;
        for (int j = 0; j < n; j++) {
            adjMatrix[i][j] = 0;
        }
    }

    printf("Enter the number of edges: ");
    scanf("%d", &edges);

    printf("Enter edges (source and destination):\n");
    for (int i = 0; i < edges; i++) {
        scanf("%d%d", &src, &dest);
        adjMatrix[src][dest] = 1;
        adjMatrix[dest][src] = 1;  // For undirected graph
    }
```

ADITYA UNIVERSITY

```
    int startVertex;
    printf("Enter the starting vertex for DFS: ");
    scanf("%d", &startVertex);

    printf("DFS Traversal:\n");
    DFS(startVertex);

    return 0;
}
```

OUTPUT:

```
Enter the number of vertices: 5
Enter the number of edges: 4
Enter edges (source and destination):
0 1
0 2
1 3
2 4
Enter the starting vertex for DFS: 0
DFS Traversal:
Visited 0
Visited 1
Visited 3
Visited 2
Visited 4


=== Code Execution Successful ===
```

ADITYA UNIVERSITY